PRJ301: Java Web Application Development

Slot 4, 5: JavaServer Pages (JSP)

VinhVT9@fe.edu.vn

Spring 2026

FPT University Can Tho

Learning Objectives

By the end of this slot, students will be able to:

- Explain **what JSP is** and why it was invented to complement Servlets.
- Understand the **JSP lifecycle**: translation, compilation, and execution.
- Use the three core JSP scripting elements: **Declarations**, **Expressions**, and **Scriptlets**.
- Identify and use **JSP Implicit Objects** (request, response, out, session, ...).
- Use **JSP Directives** (`page`, `include`, `taglib`).
- Apply **Standard Actions** (`<jsp:include>`, `<jsp:forward>`) for page dispatching.
- Combine **JSP and Servlet** in a simple web application.

FPT Education
FPT UNIVERSITY

Outline

1. What is JSP?
2. JSP Lifecycle
3. JSP Scripting Elements
4. Implicit Objects
5. JSP Directives
6. Standard Actions (Dispatching)
7. Combining JSP & Servlet
8. Classwork

FPT Education
FPT UNIVERSITY

1. What is JSP?

The Problem with Pure Servlets

Recall how we output HTML in a Servlet:

- We call `out.println("<html>...")` line by line.
- Mixing Java logic with HTML presentation is painful and hard to maintain.

The Challenge
A front-end designer cannot work on a `.java` file.
A Java developer should not care about CSS and layout details.

Solution: JavaServer Pages (JSP) — write HTML *with embedded Java*, not the other way around.

FPT Education
FPT UNIVERSITY

What is JSP?

- **JavaServer Pages (JSP)** is a server-side technology for creating dynamic web content.
- A JSP file has the extension `.jsp` and looks like an HTML file with special tags.
- It is part of the **Jakarta EE** specification (formerly Java EE).
- JSP is essentially a **higher-level abstraction** built on top of Servlets.

Key Idea
A `.jsp` file is automatically converted to a **Servlet** by the container (Tomcat).
You write HTML + Java tags; Tomcat handles the rest.

FPT Education
FPT UNIVERSITY

A Simple JSP Page

```jsp
<html>
  <body>
    <h1>Hello from JSP!</h1>
    <%
      String name = "Student";
      out.println("Welcome, " + name + "!");
    %>
  </body>
</html>
```

Listing 1: hello.jsp

- The <% ... %> block is a **scriptlet** — Java code embedded in HTML.
- out is an implicit object that writes to the HTTP response.

FPT Education
FPT UNIVERSITY

Servlet vs. JSP: When to Use Which?

Servlet	JSP
Java file (`.java`)	HTML-like file (`.jsp`)
Mainly Java code	Mainly HTML with embedded Java
Good for **controller logic**	Good for **presentation / view**
Hard to write HTML	Easy to design layout
Compiled manually	Compiled automatically by container

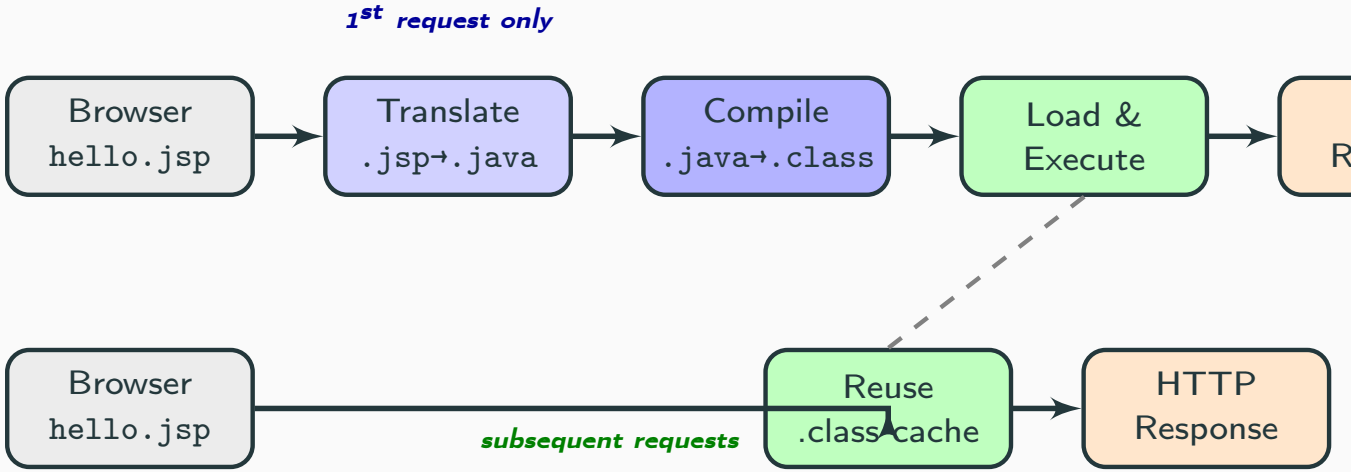
Best Practice (MVC)

Use **Servlets as Controllers** and **JSP as Views**. We will apply this in the MVC slot.

FPT Education
FPT UNIVERSITY

2. JSP Lifecycle

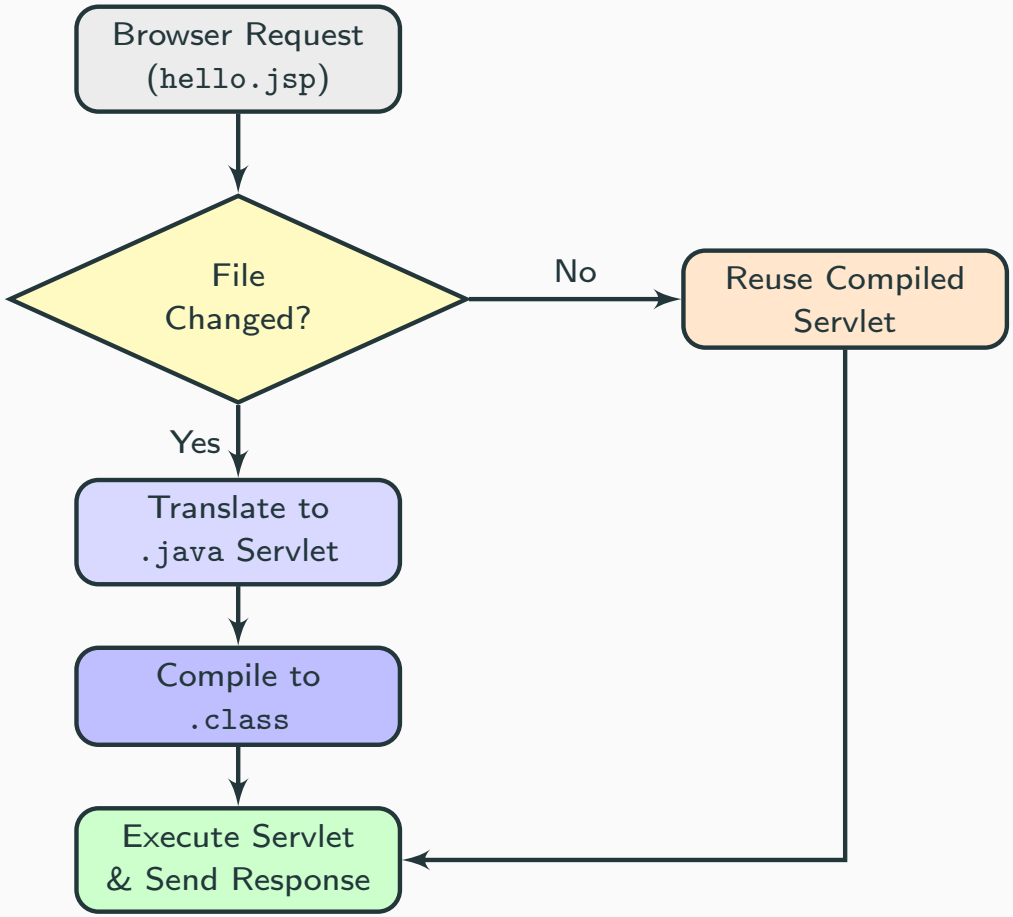
How JSP Works: The Translation Phase



Note: If the `.jsp` file *changes*, Tomcat automatically re-translates and re-compiles it (back to the top path).

FPT Education
FPT UNIVERSITY

JSP Lifecycle Diagram



FPT Education
FPT UNIVERSITY

JSP Lifecycle Methods

The generated Servlet from a JSP has these lifecycle methods:

Method	Description
`jspInit()`	Called once when JSP is first loaded (similar to `init()`)
`_jspService()`	Called for every request (similar to `service()`)
`jspDestroy()`	Called when JSP is removed from memory

Note

Your scriptlet code ends up inside `_jspService()`.

You can override `jspInit()` and `jspDestroy()` using JSP Declarations.

FPT Education
FPT UNIVERSITY

3. JSP Scripting Elements

JSP Scripting Elements Overview

There are three types of scripting elements in JSP:

Element	Syntax	Purpose
Declaration	<%! ... %>	Declare variables/methods
Expression	<%= ... %>	Output a value
Scriptlet	<% ... %>	Execute Java code

Quick Rule

<%! %> = declare <%= %> = print <% %> = execute

FPT Education
FPT UNIVERSITY

Declarations: <%! ... %>

Used to declare **class-level** variables and methods.

```
<%! int visitCount = 0; %>
<%! String siteName = "PRJ301 Demo"; %>

<%!
  public String greet(String name) {
      return "Hello, " + name + "!";
  }
%>
```

- Variables declared here are **instance variables** of the generated Servlet.
- Methods declared here can be called from scriptlets or expressions.
- End each declaration with a **semicolon**.

FPT Education
FPT UNIVERSITY

Expressions: <%= ... %>

Used to **output a value** directly into the HTML response.

```
<%! String name = "JSP World"; %>
<%! public String getName() { return name; } %>

<p>Site name: <%= siteName %></p>
<p>Method result: <%= getName() %></p>
<p>Current time: <%= new java.util.Date() %></p>
```

- The expression is **evaluated**, converted to `String`, and inserted into the page.
- Do NOT add a semicolon inside <%= ... %>.
- Expressions are evaluated **left to right**.

FPT Education
FPT UNIVERSITY

Scriptlets: <% ... %>

Used to embed **any valid Java code** into the page.

```
<%
    String username = request.getParameter("name");
    if (username == null || username.isEmpty()) {
        username = "Guest";
    }
    out.println("<p>Welcome, " + username + "!</p>");
%>
```

- Code placed here ends up inside the `_jspService()` method.
- You have access to all **implicit objects** (request, response, out, session, ...).
- Use scriptlets for **logic**: loops, conditions, data retrieval.

FPT Education
FPT UNIVERSITY

HTML Comments in JSP

JSP supports two types of comments:

```
<!-- This is an HTML comment: VISIBLE in page source -->

<%-- This is a JSP comment: HIDDEN from client --%>
```

- HTML comments are sent to the browser (visible in *View Source*).
- JSP comments are **removed at translation time** — never reach the client.
- Expressions can be embedded in HTML comments:

```
<!-- Page loaded on <%= new java.util.Date() %> -->
```

FPT Education
FPT UNIVERSITY

4. Implicit Objects

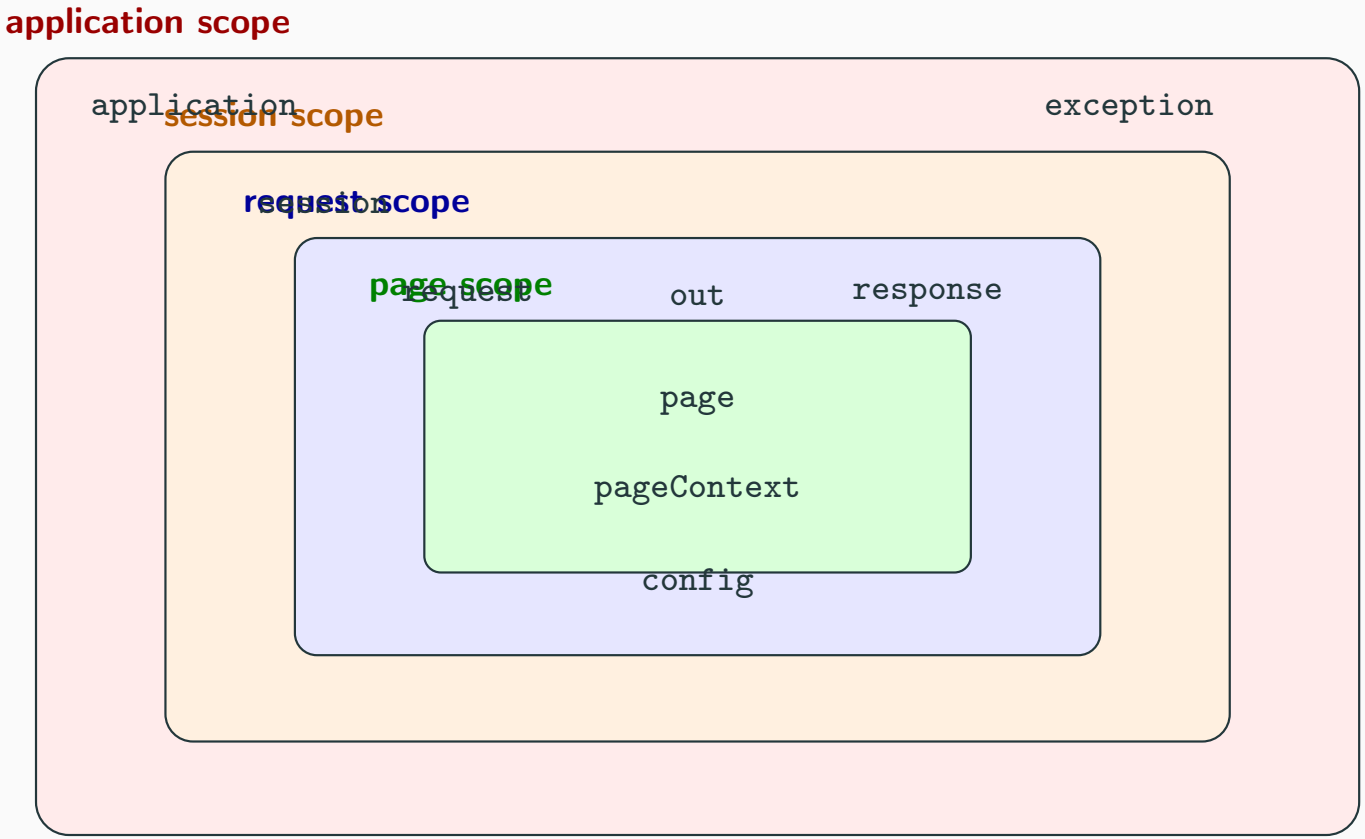
JSP Implicit Objects

The JSP container provides **9 predefined objects** available in every scriptlet/expression — no need to declare them.

Object	Type	Description
`request`	`HttpServletRequest`	The current HTTP request
`response`	`HttpServletResponse`	The HTTP response
`out`	`JspWriter`	Output stream to the client
`session`	`HttpSession`	Session for the current user
`application`	`ServletContext`	Shared across all requests/users
`config`	`ServletConfig`	Servlet configuration
`pageContext`	`PageContext`	Access to all other objects
`page`	`Object`	The JSP page itself (`this`)
`exception`	`Throwable`	Error (only in error pages)

FPT Education
FPT UNIVERSITY

JSP Implicit Objects: Scope Overview



page: current JSP only **request**: one HTTP req/resp cycle **session**: one user across requests **application**: all users, whole app

FPT Education
FPT UNIVERSITY

Using Implicit Objects: Example

```jsp
<html><body>
  <h2>Request Information</h2>
  <%
    String user = request.getParameter("username");
    String agent = request.getHeader("User-Agent");
    session.setAttribute("loggedUser", user);
  %>
  <p>Username: <%= user %></p>
  <p>Browser: <%= agent %></p>
  <p>Session ID: <%= session.getId() %></p>
  <p>App Name: <%= application.getServletContextName() %></p>
</body></html>
```

Listing 2: implicit_demo.jsp

FPT Education
FPT UNIVERSITY

5. JSP Directives

JSP Directives Overview

Directives provide instructions to the JSP container at **translation time**. They do not produce output.

Directive	Purpose
`<%@ page ... %>`	Configure the JSP page settings
`<%@ include ... %>`	Include another file at translation time
`<%@ taglib ... %>`	Declare a custom tag library

General syntax:

```
<%@ directiveName attribute="value" %>
```

FPT Education
FPT UNIVERSITY

The page Directive

Configures global settings for the JSP page.

```
<%@ page language="java"
         contentType="text/html; charset=UTF-8"
         import="java.util.*, java.sql.*"
         errorPage="error.jsp"
         isErrorPage="false"
         session="true"
         pageEncoding="UTF-8" %>
```

- **import**: Add Java imports (like `import` in a class file).
- **contentType**: MIME type of the response (default: `text/html`).
- **errorPage**: Redirect to this page on uncaught exceptions.
- Tomcat auto-imports: `javax.servlet.*`, `javax.servlet.http.*`, `java.lang.*`.

FPT Education
FPT UNIVERSITY

The page Directive: Common Attributes

Attribute	Description
`import`	Comma-separated list of Java classes/packages to import
`contentType`	MIME type + charset of the response
`session`	Whether the JSP participates in HTTP session (`true`/`false`)
`errorPage`	URL of the error-handling page
`isErrorPage`	Whether this page is an error handler
`language`	Scripting language (default: `java`)
`pageEncoding`	Character encoding of the source file

FPT Education
FPT UNIVERSITY

The `include` Directive

Includes the content of another file **at translation time** (static inclusion).

```
<%@ include file="header.jsp" %>

<h1>Main Content Here</h1>

<%@ include file="footer.html" %>
```

- The included file's content is **merged** into the current JSP before compilation.
- Changes in the included file **may not** be detected automatically — requires re-translation.
- Useful for **reusable page fragments** (headers, footers, navigation bars).

FPT Education
FPT UNIVERSITY

The `taglib` Directive

Declares a **custom tag library** (JSTL or user-defined) to be used in the page.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core"
           prefix="c" %>

<c:forEach var="item" items="${itemList}">
    <p>${item}</p>
</c:forEach>
```

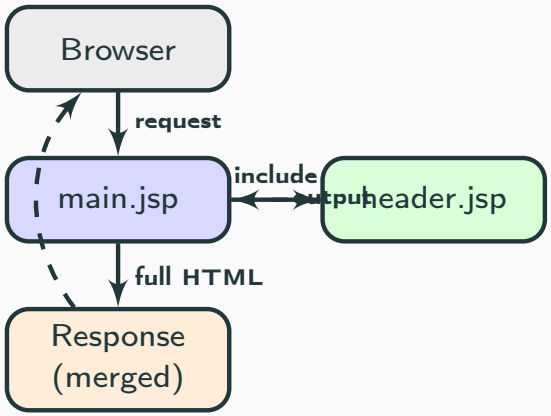
- **uri**: Identifies the tag library.
- **prefix**: The namespace prefix used in tags (`c:`, `fmt:`, etc.).
- We will cover **JSTL** in a dedicated slot.

FPT Education
FPT UNIVERSITY

6. Standard Actions (Dispatching)

Standard Actions: `include` vs `forward`

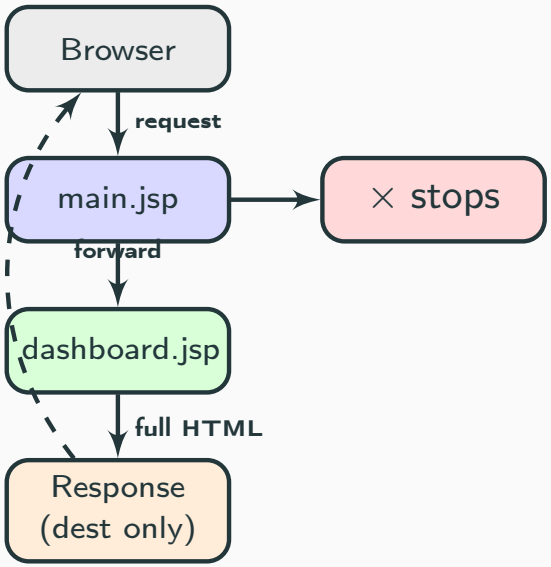
`<jsp:include>`



Both pages contribute output.

Current page **continues** after include.

`<jsp:forward>`



Only destination page outputs.

Current page output is discarded.

Key Rule

`<jsp:include>` = **merge** output entirely

`<jsp:forward>` = **hand off** control

FPT Education
FPT UNIVERSITY

<jsp:include> — Dynamic Inclusion

Includes the **response output** of another resource at request time.

```
<html><body>
  <jsp:include page="header.jsp" />

  <h1>Main Page Content</h1>

  <jsp:include page="footer.jsp">
    <jsp:param name="year" value="2026" />
  </jsp:include>
</body></html>
```

- The included page is **executed separately** and its output is merged.
- The included file can be any resource: JSP, HTML, Servlet.
- Unlike <%@ include %>, changes are **picked up immediately**.

FPT Education
FPT UNIVERSITY

<%@ include %> vs. <jsp:include>

<%@ include %>	<jsp:include>
Translation time (static)	Request time (dynamic)
Merged before compilation	Executed separately per request
Single combined Servlet	Separate Servlet per included page
Changes may require restart	Changes are immediately reflected
Faster (compiled once)	Slightly slower (runs every time)
Cannot pass parameters	Can pass <jsp:param>

Recommendation

Use <%@ include %> for **truly static** fragments (header/footer that rarely change).

Use <jsp:include> for **dynamic** content that changes per request.

FPT Education
FPT UNIVERSITY

<jsp:forward> — Forwarding Requests

Forwards the **entire request** to another resource. Processing stops on the current page.

```
<%
    String role = (String) session.getAttribute("role");
    if (role == null) {
        // Not logged in: forward to login page
%>
        <jsp:forward page="login.jsp" />
<%  } %>

<!-- This line is NEVER reached if forwarded -->
<h1>Welcome, <%= role %>!</h1>
```

- The URL in the browser **does not change** (server-side forward).
- Any output written **before** the forward will be discarded.
- Use it to implement **access control** and **conditional routing**.

FPT Education
FPT UNIVERSITY

<jsp:forward> with Parameters

You can pass additional parameters when forwarding:

```
<jsp:forward page="dashboard.jsp">
    <jsp:param name="msg" value="Login successful!" />
    <jsp:param name="userId" value="42" />
</jsp:forward>
```

- Parameters are added to the request scope.
- On `dashboard.jsp`: `request.getParameter("msg")` returns `"Login successful!"`.

Forward vs. Redirect

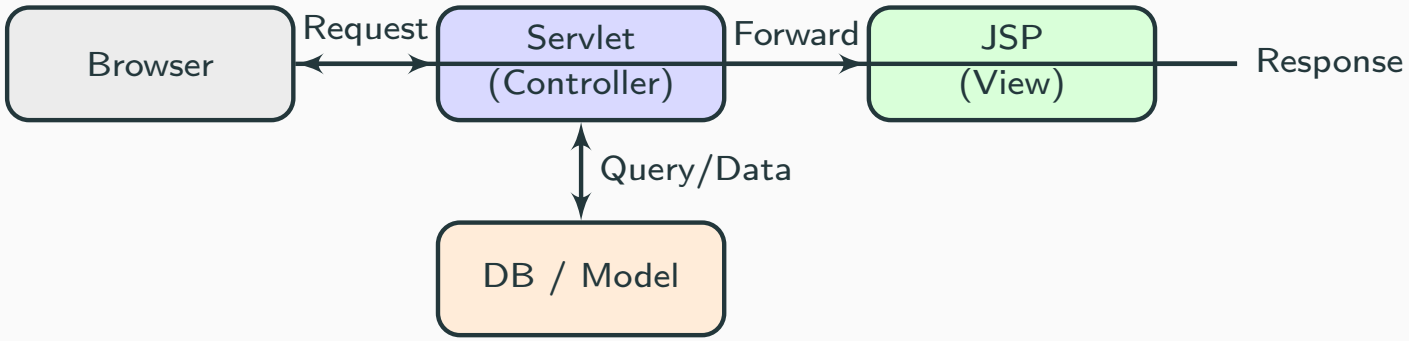
`<jsp:forward>` is a **server-side** operation — the URL does not change. `response.sendRedirect()` is a **client-side** redirect — the URL changes.

FPT Education
FPT UNIVERSITY

7. Combining JSP & Servlet

The JSP + Servlet Pattern

The most common pattern is to use **Servlet as Controller** and **JSP as View**:



1. Browser sends request to **Servlet**.
2. Servlet processes logic and places data in **request** attributes.
3. Servlet **forwards** to JSP.
4. JSP reads attributes and renders the **HTML response**.

FPT Education
FPT UNIVERSITY

Step 1: Servlet Sets Data & Forwards

```java
@WebServlet("/greet")
public class GreetServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req,
                         HttpServletResponse resp)
            throws ServletException, IOException {

        String name = req.getParameter("name");
        if (name == null) name = "Guest";

        // Place data in request scope
        req.setAttribute("greeting", "Hello, " + name + "!");

        // Forward to JSP view
        req.getRequestDispatcher("greet.jsp")
           .forward(req, resp);
    }
}
```

Listing 3: GreetServlet.java

FPT Education
FPT UNIVERSITY

Step 2: JSP Reads Data & Renders

```jsp
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<head><title>Greeting</title></head>
<body>
  <h1>
    <%-- Read the attribute set by the Servlet --%>
    <%= request.getAttribute("greeting") %>
  </h1>
  <p>Session ID: <%= session.getId() %></p>
</body>
</html>
```

Listing 4: greet.jsp

- `request.getAttribute()` retrieves data placed by the Servlet.
- The JSP only **displays** data — no business logic here.

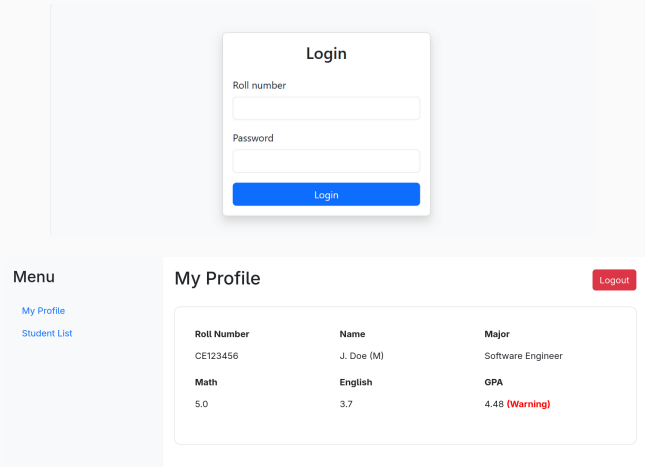
FPT Education
FPT UNIVERSITY

8. Classwork

Classwork 3: JSP Practice Tasks

Task A: JSP Scripting Elements
Create a JSP page `profile.jsp` that:

1. Declares a method `formatName(String f, String l)` that returns a full name.
2. Reads `firstname` and `lastname` query parameters.
3. Displays the formatted name and the current date/time using an expression.



Reference UI (Task B)

Task B: JSP Dispatching
Create a login form (`login.html`) that POSTs to a Servlet.
The Servlet checks credentials and either:

FPT Education
FPT UNIVERSITY

Classwork 3: Discussion Questions

1. What is the difference between a **Declaration** (<%! %>) and a **Scriptlet** (<% %>) in terms of where the generated code ends up in the Servlet?
2. Why should you **avoid** using scriptlets for business logic in a real-world application? What is the better alternative?
3. Explain the difference between `request.setAttribute()` / `request.getAttribute()` and URL query parameters. When would you use each?
4. When would you use `<jsp:forward>` instead of `response.sendRedirect()`? Give a real-world scenario.

FPT Education
FPT UNIVERSITY

Summary: Slot 4, 5 — JSP

- **What is JSP?** An HTML-centric way to build dynamic pages — compiled to Servlet by Tomcat.
- **JSP Lifecycle:** Translate → Compile → Execute (once, then cached).
- **Scripting Elements:**
  - <%! %> Declaration, <%= %> Expression, <% %> Scriptlet.
- **Implicit Objects:** `request`, `response`, `out`, `session`, `application`, ...
- **Directives:** `page`, `include`, `taglib` — configure at translation time.
- **Standard Actions:** `<jsp:include>` (dynamic), `<jsp:forward>` (routing).
- **JSP + Servlet:** Servlet handles logic → forwards to JSP for rendering.

Up Next
Slot 6, 7, 8, 9: JDBC — connecting to a database from Servlet+JSP.

FPT Education
FPT UNIVERSITY

Questions?

FPT Education
FPT UNIVERSITY